

Events

INFO

- Week TBD

Lesinhoud

Deze les gaat over het werken met events in JavaScript, hoe je event-listeners toevoegt en hoe je kunt reageren op gebruikersinteracties.

Doelstellingen

De student(e) kan:

- een logisch probleem opdelen in kleine onderdelen om tot een oplossing te komen.
- een webpagina dynamisch maken met behulp van Javascript (JS) en het Document Object Model (DOM);
- de basisprincipes van Javascript-programmatie (variabelen en constanten, controlestructuren, functies, arrays, objecten, DOM en DOM manipulatie, events en event handling) toepassen binnen een project;
- zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.

▼ Inhoud

- [Events en event-listeners](#)
- [Event-listeners toevoegen](#)
 - [Voorbeeld: Reageren op het wijzigen van de geselecteerde waarde van een select-element](#)

- [Het event object](#)
- [Event bubbling](#)
 - [Event bubbling: van binnen naar buiten](#)
 - [Stop propagation](#)
- [Veelvoorkomende event types](#)

Events en event-listeners

Webpagina's zouden nogal statisch en saai zijn als gebruikers er niet mee konden interageren. **Events** zijn de manier waarop JavaScript detecteert dat er iets gebeurt op de pagina.

Een **event** is een actie of een gebeurtenis in de browser waar je met je code op kunt reageren.

Voorbeelden van events zijn:

- De gebruiker **klikt** op een knop.
- De gebruiker **beweegt de muis** over een afbeelding.
- De gebruiker **drukt een toets in** op het toetsenbord.
- De pagina is **volledig geladen**.
- Een **formulier wordt verzonden**.

Om op deze **events** te reageren, gebruiken we **event-listeners**.

Event-listeners toevoegen

Een **event-listener** is een functie die "wacht" op een specifiek **event** op een specifiek HTML-element. Wanneer het desbetreffende event wordt gedetecteerd zal de **event-listener** de meegegeven functie uitvoeren.

De moderne en aanbevolen manier om een `event-listener` toe te voegen is met de `addEventListener()`-functie.

De syntax ziet er als volgt uit:

```
element.addEventListener('typeEvent',  
functieDieUitgevoerdZalWorden);
```

We kunnen 3 grote delen onderscheiden:

1. Het **HTML-element** waarop je wilt luisteren (bv. een knop).
2. Het **type event** als string (bv. `'click'`).
3. De **handler-functie** (ook wel `callback-functie` genoemd) die moet worden uitgevoerd wanneer het event plaatsvindt. *Een callback functie is simpelweg een functie die door een andere functie wordt opgeroepen.*

```
<button id="mijn-knop">Klik op mij</button>  
<button id="mijn-knop-anoniem">Klik op mij</button>  
<p id="bericht"></p>
```

html

```
// 1. Selecteer het element  
const knop = document.querySelector("#mijn-knop");  
const knopAnoniem = document.querySelector("#mijn-knop-anoniem");  
const bericht = document.querySelector("#bericht");  
  
// 2. Definieer de handler-functie  
function toonBericht() {  
  bericht.textContent = "Je hebt op de knop geklikt!";  
}  
  
// 3. Koppel een functie of anonieme functie aan de knop  
knop.addEventListener("click", toonBericht);  
  
knopAnoniem.addEventListener("click", () => {
```

js

```
bericht.textContent = "Je hebt op de anonieme knop  
geklikt!";  
});
```

Klik op mij Klik op mij

Merk op dat je altijd een anonieme functie of een gedefinieerde functie kunt gebruiken als handler.

Voorbeeld: Reageren op het wijzigen van de geselecteerde waarde van een select-element

We gebruiken hier het `change` event, dat specifiek wordt geactiveerd wanneer de waarde van een formulierelement, zoals een `<select>`, door de gebruiker is gewijzigd.

```
html  
<label for="kleurkiezer">Kies je favoriete kleur:</label>  
<select id="kleurkiezer">  
  <option value="">-- Maak een keuze --</option>  
  <option value="blue">Blauw</option>  
  <option value="green">Groen</option>  
  <option value="red">Rood</option>  
</select>  
  
<div id="feedback"></div>
```

```
js  
const kleurSelect =  
document.querySelector("#kleurkiezer");  
const feedbackDiv = document.querySelector("#feedback");  
  
// Het 'change' event wordt geactiveerd wanneer een
```

```
andere optie wordt geselecteerd.  
kleurSelect.addEventListener("change", () => {  
  const gekozenKleur = kleurSelect.value; // geeft de  
  value van de geselecteerde <option>.  
  
  if (gekozenKleur) {  
    feedbackDiv.innerText = `Jouw favoriete kleur is  
    ${gekozenKleur}.`;   
    feedbackDiv.style.color = gekozenKleur;  
  } else {  
    feedbackDiv.innerText = ""; // Maak de feedback leeg  
    als er geen keuze is.  
  }  
});
```

Kies je favoriete kleur: -- Maak een keuze -- ▼

Opdracht

Voeg een extra optie toe om de kleur `deeppink` te selecteren in het select-element.

Opdracht

Voeg de attributen `disabled` en `selected` toe aan de eerste optie in het `select`-element. Dit zorgt ervoor dat deze optie standaard geselecteerd is, maar dat de gebruiker deze niet opnieuw kan selecteren. Op deze manier kan je een placeholder creëren.

Het event object

Wanneer een `event` plaatsvindt, geeft de browser automatisch een `event-object` mee als eerste argument aan je `handler-functie`. Dit object bevat alle belangrijke informatie over het event dat heeft plaatsgevonden.

```
<button>Klik mij en bekijk de console</button>
```

html

```
const buttonElement = document.querySelector('button');
buttonElement.addEventListener("click", (event) => {
  console.log(event); // Bekijk dit object in de console!
});
```

js

Klik mij en bekijk de console

Enkele nuttige eigenschappen van het event object zijn:

- `event.target`: Verwijst naar het HTML-element waar het `event` oorspronkelijk op plaatsvond.
- `event.preventDefault()`: Voorkomt het standaardgedrag van de browser:
 - Het klikken op een link brengt je normaal naar een nieuwe pagina, wanneer je dit niet wilt kun je `preventDefault()` gebruiken.
 - Het indienen van een formulier herlaadt de pagina, stel dat je dit niet wilt omdat je de data via JavaScript wilt verwerken, dan kun je ook `preventDefault()` gebruiken.

```
<form id="mijn-formulier">
  <input type="text" id="naam-input" placeholder="Voer je
naam in">
  <button type="submit">Verzenden</button>
</form>
```

html

<p>Bekijk de output in de console na het indienen van het formulier.</p>

```
const formulier = document.querySelector("#mijn-formulier");

formulier.addEventListener("submit", function (event) {
  event.preventDefault(); // Voorkom standaard formulier
  gedrag (herladen van de pagina)

  const input = document.querySelector("#naam-input");
  console.log(`JavaScript kan met de data werken:
  ${input.value}`);
});
```

Bekijk de output in de console na het indienen van het formulier.

In de praktijk wordt `preventDefault()` vaak gebruikt om te voorkomen dat een formulier de pagina herlaadt wanneer het wordt verzonden, zodat je de gegevens kunt verwerken met JavaScript zonder de pagina te verlaten. Bijvoorbeeld, je kunt de gegevens naar een server (een API) sturen met behulp van `fetch()`.

Event bubbling

Wanneer een event plaatsvindt op een element, gebeurt er meer dan alleen het uitvoeren van de handler op dat specifieke element. Het event doorloopt een levenscyclus die bestaat uit drie fasen. De belangrijkste fase om te begrijpen is `event bubbling`.

Event bubbling: van binnen naar buiten

Event bubbling is het proces waarbij een event, nadat het is afgevuurd op het **target-element**, naar de parents in de DOM-structuur **bubbelt** / **borrelt** / **stijgt** / **doorgegeven wordt** / **propagateert**.

Het event wordt eerst afgehandeld op het **target**-element, dan op het **parent**-element, dan op de **parent** van het **parent**-element en zo helemaal helemaal tot boven (element -> element -> element -> ... -> document -> window).

INFO

Bekijk de console in de browser om de output van het voorbeeld te zien.

```
<div id="parent-of-parent">
  klik hier
  <div id="parent">
    klik hier
    <div class="child">Klik hier</div>
  </div>
</div>
```

html

```
div {
  padding: 1rem;
  cursor: pointer;
  color: white;
}
#parent-of-parent {
  background-color: red;
}
#parent {
  background-color: blue;
}
.child {
```

CSS


```
background-color: green;
}
```

```
const parentOfParent = document.querySelector("#parent-  
of-parent");  
const parent = document.querySelector("#parent");  
const child = parent.querySelector(".child");  
  
child.addEventListener("click", () => {  
  console.log("Je klikte op het child!");  
});  
  
parent.addEventListener("click", () => {  
  console.log("Je klikte op de parent!");  
});  
  
parentOfParent.addEventListener("click", () => {  
  console.log("Je klikte op de parentOfParent!");  
});
```



Klik op elk element in bovenstaand voorbeeld om te zien hoe het event bubbelt naar de parents.

Merk op dat:

- klikken op rood zorgt voor 1 output in de console.
- klikken op blauw zorgt voor 2 outputs in de console.
- klikken op groen zorgt voor 3 outputs in de console.

Stop propagation

Soms wil je het gedrag van `event bubbling` stoppen. Dit kan met `event.stopPropagation()`.

`console.log`

Bekijk de console in de browser om de output van het voorbeeld te zien.

html

```
<div id="parent-of-parent">
  klik hier
  <div id="parent">
    klik hier
    <div class="child">Klik hier</div>
  </div>
</div>
```

CSS

```
div {
  padding: 1rem;
  cursor: pointer;
  color: white;
}
#parent-of-parent {
  background-color: red;
}
#parent {
  background-color: blue;
}
.child {
  background-color: green;
}
```

js

```
const parentOfParent = document.querySelector("#parent-
of-parent");
```

```
const parent = document.querySelector("#parent");
const child = parent.querySelector(".child");

child.addEventListener("click", (event) => {
  console.log("Je klikte op het child!");
  event.stopPropagation(); // Stop de bubbling naar de
  voorouders
});

parent.addEventListener("click", () => {
  // Deze wordt niet uitgevoerd omdat we bubbling hebben
  gestopt
  console.log("Je klikte op de parent!");
});

parentOfParent.addEventListener("click", () => {
  // Deze wordt ook niet uitgevoerd omdat we bubbling
  hebben gestopt
  console.log("Je klikte op de parentOfParent!");
});
```



Merk op dat het `event` beschikbaar wordt via de `callback`-functie. Op dit `event`-object kan je de `stopPropagation()`-methode aanroepen om te voorkomen dat het event verder bubbelt naar de parents.

Klik op elk element in bovenstaand voorbeeld om het gedrag van `event` `bubbling` en `stopPropagation()` te zien.

Merk op dat:

- klikken op rood zorgt voor 1 output in de console.
- klikken op blauw zorgt voor 2 outputs in de console.
- klikken op groen zorgt voor 1 output in de console.

Veelvoorkomende event types

Er zijn veel verschillende soorten events. Hier is een lijst van de meest gebruikte:

Categorie	Event Type	Beschrijving
Muis-events	<code>click</code>	Wordt geactiveerd na een volledige klik (indrukken en loslaten).
	<code>mouseover</code> / <code>mouseout</code>	Wanneer de muiscursor een element binnenkomt of verlaat.
	<code>mousedown</code> / <code>mouseup</code>	Wanneer een muisknop wordt ingedrukt of losgelaten.
Toetsenbord	<code>keydown</code>	Wanneer een toets wordt ingedrukt.
	<code>keyup</code>	Wanneer een toets wordt losgelaten.
Formulier	<code>submit</code>	Wanneer een formulier wordt verzonden.
	<code>input</code>	Wordt geactiveerd telkens de waarde van een <code><input></code> , <code><select></code> , of <code><textarea></code> element wijzigt.
Focus	<code>focus</code> / <code>blur</code>	Wanneer een element focus krijgt of verliest.

Categorie	Event Type	Beschrijving
Pagina laden	<code>DOMContentLoaded</code>	Wanneer de HTML-structuur volledig is geladen en geparsed. JavaScript werd vroeger vaak binnen dit event uitgevoerd.
	<code>load</code>	Wanneer de volledige pagina, inclusief alle resources (afbeeldingen, CSS), is geladen.

Previous page
[Oefeningen](#)

Next page
[Oefeningen](#)